

Memoria Técnica: Práctica 2 - Sistemas de Objetos Distribuidos

Asignatura: Desarrollo de Aplicaciones en Red

Caso de Uso: Protocolo de votación electrónica con unicidad de cierre formal

Pareja: Mouad Harrouch Boukhalfa y Amjad A Habeeb Mahhi

Repositorio GitHub: https://github.com/mouadharr/DAR_Votacion_Electronica.git

1. Diseño de la interfaz remota

Para la evolución de nuestro sistema de votación, he diseñado la interfaz `VotacionRemota`. El objetivo principal ha sido abstraer la comunicación para trabajar directamente con la lógica de negocio, dejando de lado el envío de flujos de datos en crudo.

Las operaciones significativas definidas son:

- `emitirVoto(String dni, String candidato)`: Gestiona el registro del voto, validando la identidad (DNI) y la existencia del candidato.
- `cerrarUrna()`: Realiza el recuento final y devuelve un informe detallado con los resultados.

Este diseño eleva el nivel de abstracción, permitiendo al cliente invocar estas funciones de forma transparente y manteniendo la coherencia con el protocolo original.

2. Decisiones adoptadas

Siguiendo las directrices del anexo sobre gestión de múltiples clientes, he implementado un modelo de estado compartido. El servidor registra una única instancia del objeto remoto en el RMI Registry, lo que implica que todos los clientes interactúan sobre la misma urna electrónica.

Para garantizar la integridad de los datos frente a accesos concurrentes, he tomado las siguientes decisiones técnicas:

- **Uso de colecciones Thread-Safe:** He utilizado un `ConcurrentHashMap` para el recuento de votos y un `Collections.synchronizedSet` para el censo de votantes. Esto asegura que no existan condiciones de carrera si dos clientes votan exactamente al mismo tiempo.
- **Sincronización Crítica:** El método `cerrarUrna()` incorpora la palabra clave `synchronized`. Esto garantiza exclusión mutua, evitando que un cliente pueda emitir un voto justo en el momento en que el sistema está calculando y cerrando los resultados finales.

3. Reflexión técnica

Transformación del protocolo en métodos: En la Práctica 1 con Sockets, el protocolo era explícito. El cliente enviaba un string como "VOTAR 12345678A andrea_martos" y el servidor debía usar la función `split()` para parsear los datos. En esta práctica, el protocolo se ha vuelto implícito. El paso de mensajes se ha transformado en una llamada a método convencional, donde Java RMI se encarga del marshalling y unmarshalling de los datos de manera transparente.

Problemas que han desaparecido respecto a sockets:

- Gestión manual de mensajes: Ya no es necesario definir delimitadores de texto (como saltos de línea) ni empaquetar la información manualmente.
- Gestión de la concurrencia a bajo nivel: No he tenido que crear un bucle `accept()` ni lanzar hilos (Threads) manuales por cada cliente que se conecta, ya que el entorno RMI gestiona el pool de conexiones del servidor.
- Parsing: Ha desaparecido el riesgo de que el programa falle al trocear cadenas de texto mal formateadas.

Problemas nuevos detectados:

- Latencia de red: Aunque la sintaxis parezca una llamada a una función local, existe un retardo inherente a la red que afecta al tiempo de respuesta y de ejecución.
- Excepciones remotas: Se ha introducido la `RemoteException`. Cualquier llamada a un método remoto puede fallar si se cae la conexión o el servidor, lo que me ha obligado a implementar bloques `try-catch` en el cliente para gestionar errores de comunicación, mantener el sistema robusto y evitar cierres inesperados.

4. Instrucciones de ejecución

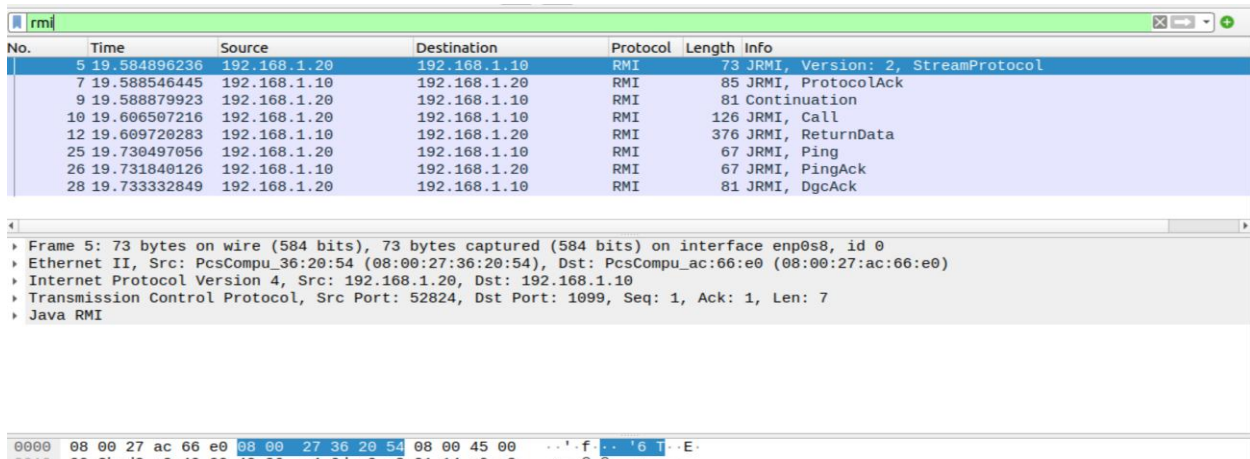
Para ejecutar y probar el sistema, sigue estos pasos en la consola:

Paso 1. Compilación: Abre una terminal en la carpeta del proyecto y ejecuta el comando:
`javac *.java`

Paso 2. Lanzamiento del Servidor: En la misma terminal, ejecuta: `java Servidor`

Paso 3. Lanzamiento del Cliente: Abre una nueva terminal en la misma carpeta y ejecuta:
`java Cliente`

5. Captura de tráfico y Ejecución distribuida



No.	Time	Source	Destination	Protocol	Length	Info
5	19.584896236	192.168.1.20	192.168.1.10	RMI	73	JRMI, Version: 2, StreamProtocol
7	19.588546445	192.168.1.10	192.168.1.20	RMI	85	JRMI, ProtocolAck
9	19.588879923	192.168.1.20	192.168.1.10	RMI	81	Continuation
10	19.606507216	192.168.1.20	192.168.1.10	RMI	126	JRMI, Call
12	19.609720283	192.168.1.10	192.168.1.20	RMI	376	JRMI, ReturnData
25	19.730497056	192.168.1.20	192.168.1.10	RMI	67	JRMI, Ping
26	19.731840126	192.168.1.10	192.168.1.20	RMI	67	JRMI, PingAck
28	19.733332849	192.168.1.20	192.168.1.10	RMI	81	JRMI, DgcAck

Frame 5: 73 bytes on wire (584 bits), 73 bytes captured (584 bits) on interface enp0s8, id 0
Ethernet II, Src: PcsCompu_36:20:54 (08:00:27:36:20:54), Dst: PcsCompu_ac:66:e0 (08:00:27:ac:66:e0)
Internet Protocol Version 4, Src: 192.168.1.20, Dst: 192.168.1.10
Transmission Control Protocol, Src Port: 52824, Dst Port: 1099, Seq: 1, Ack: 1, Len: 7
Java RMI

0000 08 00 27 ac 66 e0 08 00 27 36 20 54 08 00 45 00 ...f...6T-E-
0010 00 20 42 00 40 00 40 00 00 00 00 00 00 00 00 ...
0020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ...

Para verificar la naturaleza distribuida de la aplicación y el correcto funcionamiento del protocolo **Java RMI**, se ha procedido a realizar una captura de tráfico en el nodo **Cliente** (192.168.1.20) durante una sesión de votación completa hacia el **Servidor** (192.168.1.10).

En la imagen adjunta se observa la secuencia de mensajes intercambiados a través del puerto **1099** (puerto por defecto del RMI Registry) y los puertos efímeros negociados para la transferencia de datos:

- **Establecimiento de conexión (Handshake):** Las primeras líneas (paquetes 5 a 9) muestran el protocolo de flujo de Java RMI (StreamProtocol), donde ambos nodos sincronizan versiones y confirman la aceptación del protocolo (ProtocolAck).
- **Llamada Remota (JRMI Call):** En el paquete **nº 10**, se identifica una solicitud saliente del Cliente hacia el Servidor. Este paquete contiene la invocación del método remoto (por ejemplo, emitirVoto o cerrarUrna), incluyendo los parámetros serializados (DNI y candidato).
- **Respuesta del Servidor (JRMI ReturnData):** El paquete **nº 12** representa la respuesta del Servidor tras procesar la lógica de negocio. El tamaño del paquete (**376 bytes**) es superior al de la llamada, lo que indica el envío de información de retorno (confirmación de éxito o el recuento detallado de la urna).
- **Mantenimiento de sesión (Ping/DgcAck):** Los paquetes finales (**25 a 28**) muestran el tráfico de mantenimiento y el recolector de basura distribuido (DGC), que asegura que el objeto remoto permanezca vivo mientras el cliente mantenga la referencia activa.

La captura de wireshark está sacada de la comunicación que se puede ver en las siguientes capturas donde en la terminal del servidor se ve que está en escucha y en la terminal del cliente, por una parte, se elige la opción 1, se introduce el DNI y se elige a uno de los candidatos. En el servidor queda registrado el voto del cliente. Por otra parte, el cliente elige la opción 2 y se hace un recuento de los votos para, posteriormente, cerrar la urna. Todo eso queda registrado en la captura de wireshark comentada anteriormente.

```
Menu de votacion
1. Votar ahora
2. Ver resultados y cerrar urna
3. Salir
Elige una opcion: 1
Dime tu DNI: 12345678Z

Candidatos: a) Andrea Martos | b) Javier García | c) Pedro Gómez
¿A quien quieres votar? (a/b/c): b
Perfecto, tu voto con DNI 12345678Z se ha guardado bien.

Menu de votacion
1. Votar ahora
2. Ver resultados y cerrar urna
3. Salir
Elige una opcion: 2

Recuento final de votos
Ganador oficial: javier garcia
andrea martos: 0 votos
javier garcia: 1 votos
pedro gomez: 0 votos
```

Terminal del Cliente

```
Servidor RMI Activo. Esperando conexiones en puerto 1099...
Voto de 12345678Z para javier_garcia registrado
```

Terminal del Servidor

Conclusión técnica de la captura de Wireshark

La captura confirma que:

- **Comunicación Real:** Existe un flujo de datos bidireccional entre dos direcciones IP distintas en la misma subred.
- **Protocolo RMI:** Wireshark identifica correctamente los paquetes como **JRMI**, validando que la implementación sigue los estándares de objetos distribuidos de Java.
- **Integridad de Datos:** No se observan paquetes de retransmisión o errores TCP (RST), lo que garantiza una comunicación estable bajo la arquitectura Cliente/Servidor implementada.